

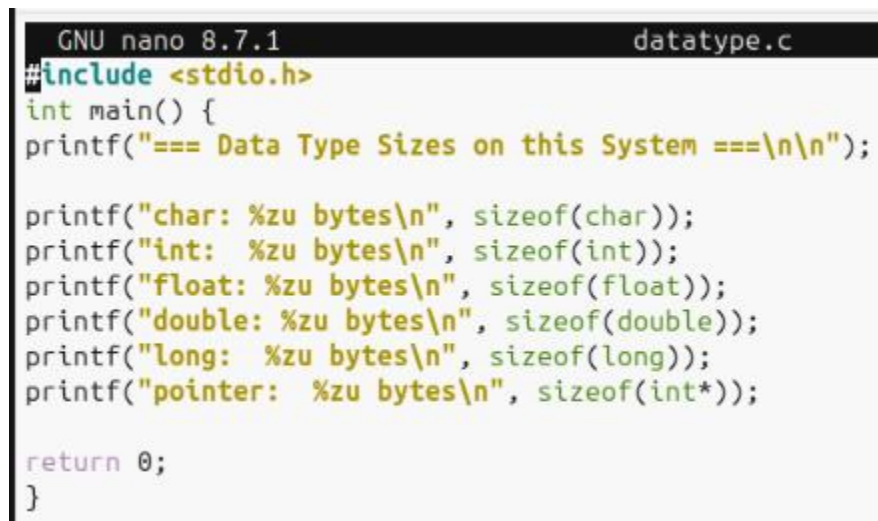
Table of Contents

Data Type Size Exploration.....	3
Summary.....	3
Memory Segment Address Mapping.....	4
Summary.....	4
Pointer and Memory Allocation.....	5
Summary.....	5
Knowledge Check Questions and Answers.....	6

Table of Figures

Figure 1: C code in datatype.c.....	3
Figure 2: Compiling and running datatype.c.....	3
Figure 3: C code in variable_address.c.....	4
Figure 4: Output of variable_address.....	4
Figure 5: C code in allocation.c.....	5
Figure 6: Compiling and running allocation.c.....	5

Data Type Size Exploration

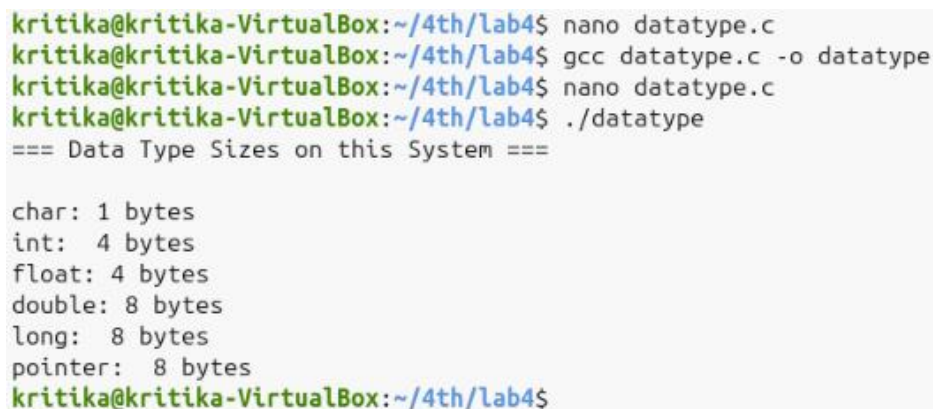


```
GNU nano 8.7.1                               datatype.c
#include <stdio.h>
int main() {
printf("=== Data Type Sizes on this System ===\n\n");

printf("char: %zu bytes\n", sizeof(char));
printf("int: %zu bytes\n", sizeof(int));
printf("float: %zu bytes\n", sizeof(float));
printf("double: %zu bytes\n", sizeof(double));
printf("long: %zu bytes\n", sizeof(long));
printf("pointer: %zu bytes\n", sizeof(int*));

return 0;
}
```

Figure 1: C code in datatype.c



```
kritika@kritika-VirtualBox:~/4th/lab4$ nano datatype.c
kritika@kritika-VirtualBox:~/4th/lab4$ gcc datatype.c -o datatype
kritika@kritika-VirtualBox:~/4th/lab4$ nano datatype.c
kritika@kritika-VirtualBox:~/4th/lab4$ ./datatype
=== Data Type Sizes on this System ===

char: 1 bytes
int: 4 bytes
float: 4 bytes
double: 8 bytes
long: 8 bytes
pointer: 8 bytes
kritika@kritika-VirtualBox:~/4th/lab4$
```

Figure 2: Compiling and running datatype.c

Summary

The program datatype.c uses sizeof() to print the size of char, int, float, double, long and a pointer. It was compiled with gcc and run on a 64-bit Linux virtual machine, and the result (1, 4, 4, 8, 8 and 8 bytes) matches the 64-bit column of Table I. long and pointers are 8 bytes here because 64-bit Linux uses the LP64 model, while a 32-bit system uses 4 bytes for both since its address registers are only 32 bits wide.

Memory Segment Address Mapping



```
GNU nano 8.7.1 variable_address.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 150;
int bss_var;

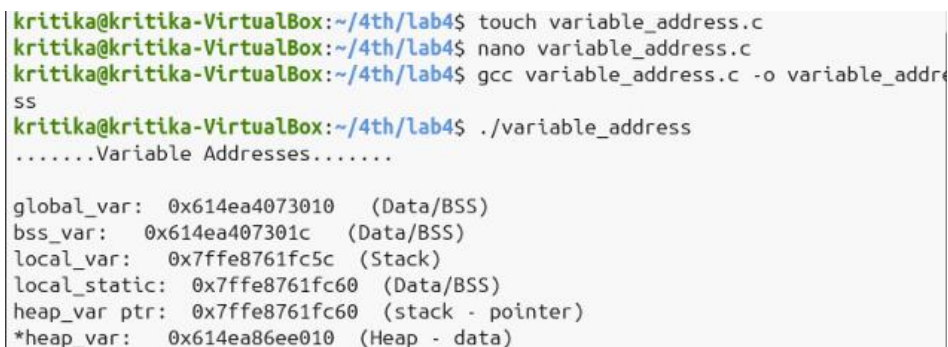
int main() {
    int local_var = 30;
    static int local_static = 20;

    int *heap_var = (int *)malloc(sizeof(int));
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var: %p (Data/BSS)\n", (void*)&global_var);
    printf("bss_var: %p (Data/BSS)\n", (void*)&bss_var);
    printf("local_var: %p (Stack)\n", (void*)&local_var);
    printf("local_static: %p (Data/BSS)\n", (void*)&local_static);
    printf("heap_var ptr: %p (stack - pointer)\n", (void*)&heap_var);
    printf("*heap_var: %p (Heap - data)\n", (void*)heap_var);

    free(heap_var);
    return 0;
}
```

Figure 3: C code in variable_address.c



```
kritika@kritika-VirtualBox:~/4th/Lab4$ touch variable_address.c
kritika@kritika-VirtualBox:~/4th/Lab4$ nano variable_address.c
kritika@kritika-VirtualBox:~/4th/Lab4$ gcc variable_address.c -o variable_address
kritika@kritika-VirtualBox:~/4th/Lab4$ ./variable_address
.....Variable Addresses.....

global_var: 0x614ea4073010 (Data/BSS)
bss_var: 0x614ea407301c (Data/BSS)
local_var: 0x7ffe8761fc5c (Stack)
local_static: 0x7ffe8761fc60 (Data/BSS)
heap_var ptr: 0x7ffe8761fc60 (stack - pointer)
*heap_var: 0x614ea86ee010 (Heap - data)
```

Figure 4: Output of variable_address

Summary

In variable_address.c the variables were placed so that they land in different segments: global_var in Data, bss_var in BSS, local_var on the stack and *heap_var on the heap. The stack has the highest addresses (0x7ffe...) and the Data/BSS area the lowest (0x614ea4...). The gap between local_var (0x7ffe8761fc5c) and *heap_var (0x614ea86ee010) is 0x1eafdef31c4c bytes, roughly 33.7 TB of virtual address space. The pointer heap_var itself is on the stack, while the memory it points to is on the heap.

Pointer and Memory Allocation

```
GNU nano 8.7.1 allocation.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *forheap;

    forheap = (int *)malloc(sizeof(int));
    *forheap = 30;

    printf("Address of pointer (Stack): %p\n", (void*)&forheap);
    printf("Address of data (Heap): %p\n", (void*) forheap);
    printf("Value stored: %d\n", *forheap);
    printf("Value stored: %d\n", *forheap);
    return 0;
}
```

Figure 5: C code in allocation.c

```
kritika@kritika-VirtualBox:~/4th/lab4$ nano allocation.c
kritika@kritika-VirtualBox:~/4th/lab4$ nano allocation.c
kritika@kritika-VirtualBox:~/4th/lab4$ gcc allocation.c -o allocation
kritika@kritika-VirtualBox:~/4th/lab4$ ./allocation
Address of pointer (Stack): 0x7ffe209ed490
Address of data (Heap): 0x61ab2d801010
Value stored: 30
Value stored: 30
kritika@kritika-VirtualBox:~/4th/lab4$ nano allocation.c
kritika@kritika-VirtualBox:~/4th/lab4$ █
```

Figure 6: Compiling and running allocation.c

Summary

In allocation.c the pointer forheap is a local variable, so its own address is on the stack (0x7ffe209ed490). The integer created with malloc lives on the heap (0x61ab2d801010), and dereferencing the pointer gives back the stored value, 30. This shows that a pointer and the data it refers to can sit in two different memory segments.

Knowledge Check Questions and Answers

1. What is a data type in the context of programming and operating systems?

In programming, a data type specifies what kind of value a variable holds (like a number, text, or true/false) and what operations you can perform on it.

In operating systems, it tells the hardware how many bytes of memory to reserve and how to interpret raw 0s and 1s into meaningful information.

2. Name the five main memory segments in a process memory layout.

- **Text:** stores the executable machine instructions of the program.
- **Data:** Initialized data holds global and static variables that have a starting value assigned in code.
- **BSS:** Uninitialized data, or bss, stores global and static variables that are not explicitly given a starting value.
- **Heap:** Heap is reserved for dynamic memory allocated at runtime when a program requests more space
- **Stack:** Stack stores local variables, function parameters, and return addresses whenever functions are called.

3. What is the difference between the Data segment and the BSS segment?

Feature	Data segment (.data)	BSS segment (.bss)
Initialization	Global/static variables initialized with non-zero values	Global/static variables that are uninitialized or initialized to zero
Executable size	Takes up space in the compiled binary on disk	Takes no space on disk; only the required size is recorded
Memory loading	Loaded into RAM from the executable at launch	Allocated in RAM at startup and zeroed by the OS
Example	static int count = 10;	static int count;

4. Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

The text segment holds the program code, which has a fixed size and does not change while the program runs, so it sits at the low end away from the regions that grow. The stack expands and shrinks as functions call each other. It is placed at the top of the address space and grows downward, while the heap starts lower and grows upward. This leaves the largest possible gap between the two, so they are unlikely to collide.

5. What happens to stack memory when a function returns? Who reclaims it?

When a function returns, its stack frame is no longer valid, but the contents of that memory are not erased. The Stack Pointer (SP) register is simply moved back to the caller's frame. The compiler generates the instructions that do this, so compiler-generated code reclaims the space, and the next function call overwrites the old data.

6. How does the OS know which memory belongs to which process?

- **Per-process page tables:** the OS keeps a separate page table for every process, mapping its virtual addresses to physical RAM frames.
- **Hardware checks:** the MMU translates every memory access using the page table of the running process.
- **Memory protection:** if a process touches an address that is not mapped in its own page table, the hardware raises a fault. The OS then sends a segmentation fault (SIGSEGV) and terminates the process.